

Prompt de création — MesDépenses

Conçois et construis depuis un dossier vide une application web complète, testée et documentée de suivi de budget personnel appelée « MesDépenses ». Le résultat doit être exécutable, sécurisé, maintenable et prêt à être repris par un autre développeur.

RÔLE ET EXIGENCES PRODUIT

Rôle : agis comme un ingénieur full-stack principal expérimenté, privilégiant architecture claire, sécurité, testabilité et maintenabilité.

Fonctionnalités : inscription, connexion et déconnexion avec JWT access/refresh ; CRUD des transactions (montant, type revenu/dépense, catégorie, date, note) ; catégories par défaut à l'inscription et catégories personnalisées ; budgets mensuels par catégorie avec alertes à 80 % et en cas de dépassement ; tableau de bord avec solde mensuel, dépenses par catégorie et tendance sur six mois ; filtrage, tri, recherche, pagination et export CSV.

SOCLE TECHNIQUE

Front-end : React, Vite, React Router, Recharts et CSS natif responsive avec variables CSS. **Back-end :** Node.js, Express et API REST ; PostgreSQL avec migrations SQL versionnées. **Qualité :** JWT, bcrypt, Zod partagé si possible, Jest, Supertest, React Testing Library, ESLint, Prettier, Docker Compose et GitHub Actions. N'ajoute aucune dépendance sans justification claire.

ARCHITECTURE ET DONNÉES

Back-end strictement organisé : **routes → middlewares → controllers → services → models/repositories**. Les controllers ne contiennent pas de logique métier ; les services ne contiennent pas de code HTTP. Règles métier — budgets, calculs et alertes — centralisées dans les services et couvertes par des tests unitaires. Réponses API uniformes : `{ data, error: { code, message, details } }`. Gestion centralisée des erreurs, logger unique et configuration validée au démarrage uniquement par variables d'environnement.

Montants stockés en centimes entiers ou `NUMERIC`, jamais en float. Chaque table possède `created_at`, `updated_at`, clés étrangères et index utiles.

EXÉCUTION PAR PHASES

Phase 0 — Plan : reformule les besoins, indique les hypothèses, propose l'arborescence finale, le schéma de données avec ERD Mermaid et le contrat API sous forme de tableau. **Phase 1 — Base de données :** migrations, données initiales et index. **Phase 2 — Back-end :** configuration, authentification, transactions, catégories et budgets, avec tests. **Phase 3 — Front-end :** layout, routing, parcours d'authentification, pages, graphiques et client API. **Phase 4 — Qualité :** lint, formatage, tests restants, Docker Compose et CI. **Phase 5 — Documentation :** README complet, scripts, endpoints, architecture Mermaid, roadmap, LICENSE et .gitignore.

SÉCURITÉ

Mots de passe hashés avec bcrypt, coût ≥ 10 ; aucun secret ni token dans les logs ; requêtes paramétrées ; validation et assainissement côté serveur ; autorisation sur chaque requête pour isoler les données utilisateur ; rate limiting sur l'authentification, Helmet, CORS strict et cookies sécurisés ; fournir `.env.example` sans secret réel.

QUALITÉ D'INTERFACE ET DE CODE

Code lisible et modulaire ; couverture minimale de 80 % sur services et routes d'authentification/transactions ; interface accessible, sémantique, navigable au clavier et contrastée ; responsive de 360 px au desktop ; états chargement, vide et erreur sur chaque page ; aucun code mort, TODO ou fichier tronqué.

RÈGLES DE TRAVAIL ET VALIDATION

Avant chaque phase, examine les cas limites : données vides, entrées invalides, mises à jour concurrentes, changement de mois, fuseau horaire, montants négatifs ou nuls. Si une exigence est ambiguë, formule une hypothèse et poursuis sans bloquer. À la fin de chaque phase, arrête-toi, résume le travail, liste les fichiers créés, indique comment lancer et vérifier le résultat, puis attends « continue ».

Avant de déclarer une phase terminée, vérifie qu'un clone propre démarre avec les commandes documentées, que toutes les entrées sont validées et autorisées, que les tests couvrent aussi les échecs, que le code respecte l'architecture et que lint/tests passent.

Définition de terminé : `docker compose up` démarre base et API, `npm run dev` démarre le front-end, et un nouvel utilisateur peut s'inscrire, ajouter une transaction, définir un budget, voir une alerte et exporter un CSV.

FORMAT DE SORTIE

Commence par la Phase 0. Pour chaque fichier, affiche son chemin complet comme titre puis son contenu intégral dans un bloc de code. N'utilise jamais de placeholder tel que « reste du code ici ». Garde les explications courtes et place-les après le code.